Arithmetic Expression Evaluator

Test Cases
Version 1.0

Winston Industries LLC

Date	Version	Description	Author
7/May/2026	**1.0**	**Tested and documented all 25 test cases**	**Caden Gudenkauf**

Revision History

Date	Version	Description	Author
07/May/2026	1.0	Initial draft - all 25 test cases authored	Caden

Table of Contents

Test Case

1. Purpose

This Test Case Specification document for the Arithmetic Expression Evaluator (AEE), developed by Winston Industries LLC for EECS 348, defines the test cases used to validate each functional and non-functional requirement. Tests cover the Tokenizer, Parser, Evaluator, and ErrorHandler components and are designed to be repeatable across all levels.

2. Test case identifier

Test cases are identified with the prefix TC followed by a two-digit sequence number (TC01 through TC25). Identifiers are unique across all modules. The full table in Section 4 maps each identifier to its module, description, test data, expected result, actual result, and pass/fail status.

3. Test item

The system under test is a C++ Arithmetic Expression Evaluator that accepts an expression, tokenizes it into Token objects, converts the token to Reverse Polish Notation, evaluates the RPN using a stack, and returns a double result. Error codes are converted to user friendly messages by the ErrorHandler.

Tokenizer (tokenizer.cpp) - Analyses and producesToken objects.

Parser (parser.cpp) - Shunting-Yard conversion with precedence and error detection.

Evaluator (evaluator.cpp) - RPN evaluation returning a double result.

ErrorHandler (error_handler.cpp) - Maps ErrorCode values to readable messages.

4. Input specifications

Each test case provides a pre-built vector of Token objects, bypassing the tokenizer (matching the pattern used in unit_tests.cpp). Inputs include: positive and negative, the five operators (+, -, *, /, %, **), and left/right parentheses. All test cases are listed in the table below. Actual result and pass/fail columns are filled during execution.

TC ID	Module	Description	Test Data / Input	Expected Result	Pass/ Fail
TC01	Tokenizer	Token construction: numeric literal	Token num("42")	token="42", prec=0, assoc=' ', type=NUM	PASS
TC02	Tokenizer	Token construction: negative number	Token neg("-7")	token="-7", type=NUM	PASS
TC03	Tokenizer	Operator token stores correct precedence/associativity	Token op("**",3,'R',OP)	prec=3, assoc='R', type=OP	PASS
TC04	Tokenizer	Parenthesis token types assigned correctly	Token lp("(",LPAREN); Token rp(")",RPAREN)	lp.type=LPAREN, rp.type=RPAREN	PASS
TC05	Tokenizer	Token equality operator returns true for identical tokens	Token a("+",1,'L',OP) == Token b("+",1,'L',OP)	true	PASS
TC06	Parser	RPN output: complex valid expression (sample)	(14 + 16 - 12) * 3 / -4 % 6 ** 2	14 16 + 12 - 3 * -4 / 6 2 ** %, error=NONE	PASS

TC ID	Module	Description	Test Data / Input	Expected Result	Pass/ Fail
TC07	Parser	RPN output: single number token	42	42, error=NONE	PASS
TC08	Parser	Left-associativity: same-precedence pop left first	1 - 2 + 3	1 2 - 3 +, error=NONE	PASS
TC09	Parser	Right-associativity of exponent operator	2 ** 3 ** 4	2 3 4 ** **, error=NONE	PASS
TC10	Parser	Nested parentheses handled correctly	((1 + 2))	1 2 +, error=NONE	PASS
TC11	Parser	Multiplication has higher precedence than addition	1 + 2 * 3	1 2 3 * +, error=NONE	PASS
TC12	Parser	Parentheses override default operator precedence	(1 + 2) * 3	1 2 + 3 *, error=NONE	PASS
TC13	Parser	Mixed precedence with exponent: (2+3)*(4-1)**2	(2 + 3) * (4 - 1) ** 2	2 3 + 4 1 - 2 ** *, error=NONE	PASS
TC14	Parser	Error: missing opening parenthesis	12 + -4)	error=MISMATCHED_PARENTHESIS	PASS
TC15	Parser	Error: missing closing parenthesis	(3 - 2	error=MISMATCHED_PARENTHESIS	PASS
TC16	Parser	Error: adjacent operators	14 + + 3	error=ADJACENT_OPERATORS	PASS
TC17	Parser	Error: operator after opening parenthesis	(/ 1	error=INVALID_SYNTAX	PASS
TC18	Parser	Error: operator before closing parenthesis	(12 +)	error=INVALID_SYNTAX	PASS
TC19	Parser	Error: operator at start of expression	+ 5	error=INVALID_EXPRESSION	PASS
TC20	Evaluator	Evaluate simple addition	RPN: 3 4 +	7.0	PASS
TC21	Evaluator	Evaluate subtraction yielding negative result	RPN: 3 10 -	-7.0	PASS
TC22	Evaluator	Evaluate integer division with float result	RPN: 7 2 /	3.5	PASS
TC23	Evaluator	Error: division by zero	RPN: 5 0 /	error=DIVIDE_BY_ZERO	PASS
TC24	Evaluator	Evaluate exponentiation (2^10)	RPN: 2 10 **	1024.0	PASS
TC25	Evaluator	Evaluate modulo operation	RPN: 10 3 %	1.0	PASS

5. Output specifications

RPN token vector - an ordered vector of Token objects representing postfix form. Verified by using the overloaded Token == operator.

ErrorCode value - a non-NONE ErrorCode in the ParserResult/EvaluatorResult struct. Error tests pass when the returned code matches exactly.

Double value - the numeric result of evaluating an RPN expression.

Console output - for ErrorHandler tests, a non-empty string printed to stdout isa pass.

6. Environmental needs

6.1.1 Hardware

No special hardware required. Any modern PC capable of compiling and running C++ code is sufficient.

6.1.2 Software

Compiler: g++ with C++17 or later. Build system: make (see src/makefile). OS: Windows (primary, .exe included) or any POSIX system. No third-party testing framework required; tests are driven by unit_tests.cpp and the Unit_Tests() entry point.

6.1.3 Other

No additional requirements.

7. Special procedural requirements

1. Build the project: navigate to src/ and run make. Confirm compilation succeeds with no errors.

2. Run parser, evaluator and tokenizer tests by invoking Unit_Tests() from main.cpp, which calls RUN_PARSER_TEST() defined in unit_tests.cpp.

3. Record actual results and pass/fail status in the table in Section 4 after each run.

8. Intercase dependencies

TC01-TC05 (Tokenizer): no dependencies; may run in any order.

TC06-TC19 (Parser): depend on TC01-TC05, as the Parser receives Token objects that must be verified first.

TC20-TC25 (Evaluator): depend on TC06-TC13, as the Evaluator receives the correct RPN from the Parser.